Graph-of-Code Python (GoCP)

A systematic report on a proposed code clone detection method using intelligent software engineering techniques

Ilya Sullivan

Submitted: April 2026

Abstract

Duplicate functions in large programs are a persistent challenge for software maintenance and can cause inflated sizes. Traditional search and analysis tools are able to detect similar code functions but fail when implementation is semantically different. This paper presents Graph-of-Code Python (GoCP), an adaptation of the Graph-of-Code approach to clone detection in the Python language.

GoCP represents each function as a 56-dimensional graph fingerprint; function pairs are compared using a 229-dimensional pairwise structural feature vector. The model is trained with the feature vectors using a Histogram-based Gradient Boosting Classifier and evaluated against three other models: Jaccard Similarity, TF-IDF, and TF-IDF restricted to Python keywords. In total, ~100,000 function pairs were used in the training using 10-fold stratified cross-validation, resulting in GoCP having the highest accuracy of 0.8864 and is the only model that rejects non-clones while also detecting clones across all types. GoCP demonstrates that the fingerprinting method can generalise, maintaining partial semantic clone detection beyond what token-based approaches achieve.

1 Introduction

Functions that serve the same purpose as each other can occur in a variety of different stages of software development, from design, programming, debugging, etc. for every new duplicate function made, the size and maintainability for the program increases in complexity, acting as a hindrance for any future developers especially when a bug exists in them [1], [2].

Code clone detection has been studied extensively, from token-based matching [3] to abstract syntax tree (AST) comparison [4]; these solutions mainly detect surface-level similarity, struggling with semantic clones (Type 3 and Type 4 functions), where two functions are implemented differently but logically do the same things.

There are four types of code clone functions [5]:

- **Type I**: Identical code fragments except for variations in whitespace (may be also variations in layout) and comments.
- **Type II**: Structurally/syntactically identical fragments except for variations in identifiers, literals, types, layout and comments.
- **Type III**: Copied fragments with further modifications. Statements can be changed, added or removed in addition to variations in identifiers, literals, types, layout and comments.
- **Type IV**: Two or more code fragments that perform the same computation but implemented through different syntactic variants.

This paper builds off the work of Alhazami E. and Sheneamer A. [6], who proposed the Graph-of-Code (GoC) approach for clone detection using dependency graphs-theoretic features. Unlike token-based methods, GoC does not rely on identifier names during comparison, which makes it particularly useful for codebases where exposing variable and function names could reveal proprietary algorithms and sensitive code. Graph-of-Code Python re-implements this approach in Python from the Java implementation analysed in their paper.

GoCP has been analysed against three baselines: Jaccard Similarity — as the baseline to beat; TF-IDF (keywords only) — for a comparable alternate model; TF-IDF (full tokens) — for a comparison for a popular model used in code clone detection. Each model has been tested on 100,000 function pairs sampled from the dataset PoolC [7], using 10-fold stratified cross-validation using five metrics: accuracy, precision, recall, F1, and AUC-ROC. The dataset was chosen due to its high pair count and its simple data parsing. Graph-of-Code Python was found to outperform Jaccard Similarity on F1 and is on par to TF-IDF (keywords only) [Figure 3]. While TF-IDF (Full) achieves a higher F1, this is almost entirely due to its recall bias, favouring positive clone detection but classifying all false pairs as clones in qualitative evaluation [Figure 2]. GoCP achieves the most balanced precision-recall trade-off across all clone types [Figure 2].

The contributions of this paper are: (1) a Python re-implementation of the Graph-of-Code approach proposed by E. A. Alhazami and A. M. Sheneamer [6] adapted for dynamic typing and more flexible syntax; and (2) a 229-dimensional pairwise structural fingerprint that compares dependency graph properties without exposing identifier information.

The paper is structured as follows: Section 2 surveys related work on the different approaches to code clone detection; Section 3 proposes the GoCP solution; Section 4 describes the experimental setup, dataset, metrics for evaluation and baselines used to compare GoCP; Section 5 presents the results supported by the figures in Section 10; Section 6 looks at the limitations of GoCP and how those can be seen in the results.

2 Related Work

Token-based approaches to code clone detection tokenise source code and compare token sequences, detecting Type-1 & 2 clones effectively [3], [8]. This strategy fails when statements are added, removed, or reordered (Type 3), or when the functions are implemented differently (Type 4), due to only detecting on similarity metrics that analyse how code is written not what the code computes.

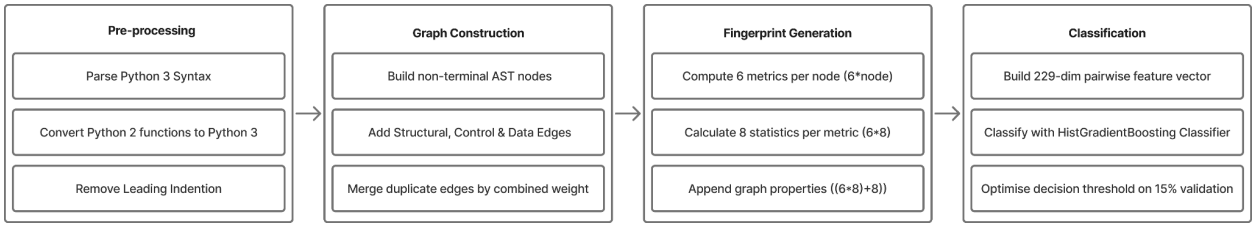
AST-based approaches parse code into syntax trees, allowing for structure comparisons between two functions, extending coverage to Type-3 clones [4], [9], [10]. Implementation using ASTs limits the analyses to only detect for similar syntactic structures, failing for Type-4 clones.

Graph-based approaches achieve partial Type-4 coverage by graphing relationships as edges and operations as nodes [4], [6]. By implementing it using control and data dependencies, functions which are implemented differently will have similar graph representations. The Graph-of-Code approach extends this idea by combining structural, control, and data-dependency edges, although comes at a greater computational cost than token- or AST-based methods.

Machine Learning (ML) approaches use classification for code clone detection [11]. Methods range from vectorisation such as TF-IDF [12] to deep-learning models [13], are able to achieve high accuracy on benchmarks but are sensitive to token vocabulary of their training data, providing no structural awareness of underlying code [11]; Performance degrades on out-of-distribution code and Type-4 clones where surface similarity is absent.

3 Solution

GoCP uses a four stage system: pre-processing; graph construction; feature extraction; and classification.



3.1 Preprocessing

In preprocessing, each function is normalised; First, the functions are passed through a parser which checks for valid Python 3 syntax — if this fails, it is assumed that the given function is Python 2, and conversion is attempted, replacing constructs such as adding brackets to open print statements and converting spaced operators to its non-spaced counterpart. The next step removes leading indentation to handle functions copied from within a class. Without these two parsing steps, many functions which would otherwise be valid training data would silently be excluded or fall back to raw text comparison rather than graph-based analysis.

3.2 Graph Construction

The data is constructed into an AST for the non-terminal nodes only, filtering out leaf nodes such as names and constants as they encode identifier-level information rather than structural meaning and therefore would reduce the performance of the graph metrics.

Structural edges are first added, representing the parent-child relationship of the non-terminals in the function, such that every node has exactly one parent without cycles and no two nodes have a child in common. As structural edges alone form a tree, it contains no triangles and therefore the clustering coefficient and transitivity would always be zero, so control and data dependencies are added as edges to enable the model to learn better patterns about the

function. Control dependency edges are added from each control node to all of its non-terminal descendants. Data dependency edges connect variable definitions to every subsequent use of that variable, helping to capture data flow through the function and enabling detection of functions that implement the same logic through structurally different code. In scenarios where two edges connect to the same two nodes, the edge weights are combined and a single edge is maintained to preserve graph topology.

3.3 Graph Fingerprint Generation

A fixed-length 56-feature fingerprint is produced from each graph, representing the structural and topological properties of the function. These features are gathered in two steps: node-level analysis and graph-level metrics, creating a 56-feature vector per function as defined by GoC [6].

Six node-level metrics are calculated for every node in the graph including: weighted degree; Eigenvector centrality; PageRank; Two-hop Count; Local Clustering Coefficient; and Neighbour Clustering. Due to each function having a varying number of nodes, each metric is reduced to eight summary statistics: median, mean, standard deviation, variance, skewness, mode, minimum, and maximum. This produces 48 node-level features that are able to capture a function of any length. Eight additional graph-level metrics are added to the feature set with the aim to capture any other trends that the node-level aggregates can't express — including: total edges; total triangles; network size; total weighted degree; density; assortativity; transitivity; and average shortest path length.

The pairwise feature is constructed differently to GoC [6], by instead comparing the measures between the two 56-dimensional fingerprints by using the following measures: absolute element-wise difference $|va - vb|$, indicating the true divergence at each graph metric (56 dimensions); element-wise Canberra distance $\frac{|va-vb|}{|va|+|vb|+\varepsilon}$, indicating the relative divergence at each graph metric, helping the model compare features across different scales (56 dimensions); proportionality ratio $\frac{min(|va|,|vb|)}{max(|va|,|vb|)}$, to capture how each dimension scales between the two functions (56 dimensions); a binary indicator to indicate whether both fingerprint values are non-zero values as without the flag (0.7,0.7) and (0,0) would indicate the same (56 dimensions); a size ratio that signals the graph size mismatch $\frac{min(netsize_a, netsize_b)}{max(netsize_a, netsize_b)}$, helping the classifier distinguish by number of nodes (1 dimension); and four global similarity metrics — Canberra, Cosine, Euclidean and Pearson Correlation. The function fingerprints are not given to the classifier to prevent the model from learning domain identity, but rather learn the structural similarity.

3.4 Classification

A Histogram-based Gradient Boosting Classifier is used to train the model, chosen for its robustness to mixed-scale features such as the 229-dimensional pairwise vector. Class weighting is applied to ensure that if there are an unequal number of clone to non-clone pairs, the classifier will not take bias toward the class with the majority; although an equal number of clone and non-clone pairs were selected during preprocessing, errors during the steps may cause some pairs to be excluded for a variety of reasons. Early stopping is enabled, a maximum of 500 iterations allowed, preventing overfitting without additional tuning.

The decision threshold was learned from a 15% validation subset of the dataset, maximising for F1 — this decision was made to account for domain characteristics and class imbalance found in the training data.

4 Setup

4.1 Dataset

GoCP is trained using the PoolC dataset [7] and the CodeSearchNet dataset [14], [15].

PoolC is a dataset with 6,712,982 function pairs, with an equal split of negative and positive clones. The selection of using it was due to its scale, simple structure and straightforward pair-labelled structure, all of which stood out from other alternatives found.

100,000 pairs were sampled (~50,000 positive, and ~50,000 negative) due to hardware constraints, training taking ~25 minutes. To mitigate same-domain bias from training exclusively on competitive programming code, 70% of negative pairs were drawn from CodeSearchNet, totalling 35,000 of the negative pairs. CodeSearchNet is a Python corpus that has sourcing functions from a variety of different publicly available GitHub projects, better reflecting the diversity of real-world Python code compared to PoolC. As CodeSearchNet provides unlabelled functions only, without clone pair annotations, it could not be used as a positive training source.

4.2 Evaluation Metrics

GoCP uses the standard evaluation metrics to evaluate the model across 10-fold stratified cross-validation: Accuracy, Precision, Recall, F1, and AUC-ROC, as well as end-to-end prediction time; Wilcoxon signed-rank testing is used to assess if the observed differences are statistically significant.

To further aid in testing and evaluation of the models, a set of 50 test cases were created — 10 per clone type plus 10 false pairs to represent the four clone types, as described in section 1. Each case was created to cover a range of algorithmic patterns that cover sorting, search, and recursion, to assess where the models struggle. By using a controlled environment instead of the dataset, the areas the model succeeds and fails are more transparent. [Figure 1]

4.3 Baseline

To evaluate the performance of the model, it is compared to three other models: Jaccard Similarity; TF-IDF (Full); TF-IDF (Keywords only).

Jaccard Similarity has been chosen as the primary baseline to beat, using the simplest measure of code similarity by comparing the proportion of shared tokens between two functions. A method that cannot outperform simple token overlap does not justify the additional complexity that comes with it.

TF-IDF (Full) and TF-IDF (Keywords) both represent token-based machine learning baselines, trained using a Random Forest Classifier (RFC) on a pairwise feature matrix of $(v_a, v_b, |v_a - v_b|)$ with a vocabulary of up to 2000 tokens. RFC was chosen over choosing the same Histogram Gradient Boosting (HGB) that GoCP used as HGB handles mixed-scale continuous features better, while RFC is the standard for sparse TF-IDF vectors. The full model uses all identifiers, operators, and keywords, whilst the keywords model restricts the vocabulary to Python keywords only in order to reduce surface-level bias learnt through variable name similarities. The TF-IDF models are trained on exclusively PoolC rather than including the CodeSearchNet negatives in order to not introduce general-purpose Python tokens that would alter the feature distribution negatively, and cause a mismatched feature space.

5 Results

[Figure 3] displays the cross-validation results across all five metrics as described in section 4.2 . Graph-of-Code Python achieves the highest accuracy (0.8864), outperforming all three baselines. Wilcoxon signed-rank tests across 10-folds confirm the accuracy advantage of GoCP compared to the other three baselines (w=0, p=0.002, 0.98). TF-IDF (Full) records the highest F1 (0.9222) however the recall is also highest across all models (0.9962), showing that it predicts almost every pair as a clone regardless of if true. complete failure to detect non-clones in the qualitative test [Figure 1] — TF-IDF (Full) classifies all false pairs as clones, scoring 0/10 on false pair rejection. Jaccard's recall of 1.0000 with zero standard deviation indicates it classifies every pair as a clone without exception. GoC and TF-IDF (Keywords) have similar F1 however GoCP's AUC-ROC of 0.9510 supersedes TF-IDF (Keywords) at 0.8219, demonstrating that GoC can more effectively discriminate between clones and non-clones.

[Figure 1] demonstrates where each model fails and succeeds by clone type. For types 1 and 2, all methods perform well, with all but Jaccard scoring 10/10 for all [Figure 2]. Jaccard fails to identify all clones for type 2, scoring 1/10, likely due to the fundamental problem with comparing the similarity of code by tokens alone as any renames of identifiers can significantly affect the probability of being detected. GoCP succeeds in this area, ignoring all identifiers and relying on the structure of the code. This structural independence does however make GoCP sensitive to slight changes to the code, as on Type 3, GoCP scores 6/10, faltering on additional statements which affects the graph and therefore the fingerprint for the function to the point to fall below the detection threshold even whilst the structure remains largely the same. In comparison the other models all succeed in determining the code clones for Type 3. On Type 4, GoCP detects 4/10 of the code clones, expected for Type 4 due to the graphical structures sometimes looking different as implementation differs. Jaccard Similarity fails to detect clones due to having little similar tokens to determine if it is the same, scoring 0/10. Both TF-IDFs detect all Type 4 clones correctly, scoring 10/10 but it becomes apparent that this is a consequence of their poor discrimination rather than semantic understanding — both models fail to reject false pairs, scoring 0/10, meaning they predict almost everything as a clone regardless of if true. GoCP scores 4/10 on false pairs, the only method to reject non-clones whilst achieving partial Type 4 clone detection, indicating that its structural comparison is able to generalise beyond the training set.

End-to-end prediction time is found to be fastest for GoCP compared to both TF-IDF baselines (109.84ms vs 143-147ms), even despite its additional graph construction. The higher variance of GoCP demonstrates that the time to predict is dependent largely on the graph building time when the function is larger.

Overall, GoCP is the most balanced method across all clone types and while it does not achieve the highest F1, it leads in accuracy, and is the only method capable of both detecting structural clones and reliably rejecting non-clones.

6 Reflection

GoCP re-implements the Graph-of-Code approach in Python, but whilst it demonstrates competitive performance against token-based baselines, a number of factors limit the results compared to the original performed by E. A. Alhazami and A. M. Sheneamer [6]

The most significant difference between GoC and GoCP is the difference in datasets. The original GoC was evaluated on BigCloneBench, a large-scale Java benchmark, containing 7,868,560 clone pairs that are explicitly labelled by clone type across 25 open-source real-world projects [6]. GoCP on the other hand trains on 100,000 pairs sampled from PoolC, providing no clone type and consists entirely of competitive programming functions. This problem is partially mitigated by using an additional dataset, CodeSearchNet; the largest problem is the lack of real project functions that would appear in a real case — the result is a weaker Type 3 and Type 4 detection. The 100,000 training cap was caused by hardware constraints but would likely not improve significantly further due to the issues described above.

Beyond this, it should be noted that the implementation of using graph structure as the way to determine code clones causes there to be a ceiling for what it can do, with large functions that implement the structure differently but logically the same, hard to determine using structure alone — this means that there is a ceiling for what it can detect.

To further limit GoCP's performance relative to GoC, it uses Python, a language that uses dynamic typing and flexible syntax, presenting a structural challenge to determine code clones. This particularly affects the control-dependency edges that GoC relies on, inconsistently mapping Python constructs and reducing fingerprint stability, with slight changes affecting the end fingerprint produced.

By using a calculated decision threshold, it attempts to work on the precision-recall tradeoff. By setting the threshold too low, some false pairs will be seen to be Type-4 clones and be falsely accepted but setting it too high will cause some Type-4 clones to be calculated as false clones. The decision threshold of GoCP was calculated to be 0.44 but analysing some of the evaluation tests, the tests “Kadane vs is_prime” (0.45) and “Palindrome check vs bubble sort” (0.51) are falsely reported as clones but are close to the threshold, suggesting they sit in an ambiguous region where

the graph structures are coincidentally similar despite not being clones. A per-type threshold would allow this balance to be better adjusted.

7 Conclusion

GoCP is a Python adaptation of the Graph-of-Code approach to code clone detection [6], representing functions as graphs, classifying pairs using a 229-dimension pairwise structural fingerprint. It is evaluated against three other baseline models, trained using 100,000 function pairs from PoolC [7] and CodeSearchNet [15]. GoCP achieves an accuracy of 0.8864, the highest tested. Although it fails to have the highest F1 score, its ability to better learn the structure of code clones over exploiting recall bias makes it a more reliable model compared to the baselines.

GoCP demonstrates that fingerprinting is a method that can be used for code clone detection with some success corroborated by the original paper on Java. Where token-based approaches fail, graph-based approaches like GoC can be used to detect clones albeit with a performance ceiling for Type-4 clones due to the structurally divergent implementations.

Further research should be put on creating or testing GoC on a labelled Python clone dataset similar to that of BigCloneBench to address the Type 3 and Type 4 weaknesses found in evaluation of GoCP.

8 Artifacts

GitHub Links

- Source Repository: https://github.com/IlyaSulli/GoCP
- Requirements: https://github.com/IlyaSulli/GoCP/blob/main/requirements.pdf
- Manual: https://github.com/IlyaSulli/GoCP/blob/main/manual.pdf
- Replication: https://github.com/IlyaSulli/GoCP/blob/main/replication.pdf
- Raw Results: https://github.com/IlyaSulli/GoCP/tree/main/results

9 References

[1] G. Shobha, A. Rana, V. Kansal and S. Tanwar, "Comparison between Code Clone Detection and Model Clone Detection," 2021 9th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO), Noida, India, 2021, pp. 1-5, doi: 10.1109/ICRITO51393.2021.9596454.

[2] P. V. Bhaskar and Geetika, "Code Clone Detection: Techniques for Revealing Code Clones," 2025 5th International Conference on Pervasive Computing and Social Networking (ICPCSN), Salem, India, 2025, pp. 1414-1420, doi: 10.1109/ICPCSN65854.2025.11035972.

[3] Y. Yuan and Y. Guo, "Boreas: an accurate and scalable token-based approach to code clone detection," 2012 Proceedings of the 27th IEEE/ACM International Conference on Automated Software Engineering, Essen, Germany, 2012, pp. 286-289, doi: 10.1145/2351676.2351725.

[4] Z. Swilam, A. Hamdy and A. Pester, "Cross-Language Code Clone Detection Using Abstract Syntax Tree and Graph Neural Network," 2023 International Conference on Computer and Applications (ICCA), Cairo, Egypt, 2023, pp. 1-5, doi: 10.1109/ICCA59364.2023.10401783.

[5] C. K. Roy and J. R. Cordy, “A survey on software clone detection research,” Queen’s Sch. Comput. TR, vol. 541, no. 115, pp. 64–68, 2007.

[6] E. A. Alhazami and A. M. Sheneamer, "Graph-of-Code: Semantic Clone Detection Using Graph Fingerprints," in IEEE Transactions on Software Engineering, vol. 49, no. 8, pp. 3972-3988, Aug. 2023, doi: 10.1109/TSE.2023.3276780.

[7] PoolC, "1-fold-clone-detection-600k-5fold," Hugging Face, 2022. [Online]. Available: https://huggingface.co/datasets/PoolC/1-fold-clone-detection-600k-5fold

[8] T. Kamiya, S. Kusumoto and K. Inoue, "CCFinder: a multilinguistic token-based code clone detection system for large scale source code," in IEEE Transactions on Software Engineering, vol. 28, no. 7, pp. 654-670, July 2002, doi: 10.1109/TSE.2002.1019480.

[9] H. He, L. Lin, T. Yu and X. Zhong, "CloneBAS: A Code Clone Detection Method Based on Abstract Syntax Tree and Simhash," 2023 IEEE 3rd International Conference on Data Science and Computer Application (ICDSCA), Dalian, China, 2023, pp. 1539-1544, doi: 10.1109/ICDSCA59871.2023.10392292.

[10] Lingxiao Jiang, Ghassan Misherghi, Zhendong Su, and Stephane Glondu. 2007. DECKARD: Scalable and Accurate Tree-Based Detection of Code Clones. In Proceedings of the 29th international conference on Software Engineering (ICSE '07). IEEE Computer Society, USA, 96–105. https://doi.org/10.1109/ICSE.2007.30

[11] Z. Bano and M. Z. Ansari, "Code Clone Detection using Machine Learning: Brief Overview and Latest Developments," 2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT), Kamand, India, 2024, pp. 1-6, doi: 10.1109/ICCCNT61001.2024.10725802.

[12] Z. Bano, A. Kamal and M. Z. Ansari, "Combining TF–IDF and Random Forest Techniques for Code Clone Detection," 2025 4th International Conference on Automation, Computing and Renewable Systems (ICACRS), Pudukkottai, India, 2025, pp. 727-732, doi: 10.1109/ICACRS67045.2025.11324259.

[13] G. Li, Y. Tang, X. Zhang and B. Yi, "A Deep Learning Based Approach to Detect Code Clones," 2020 International Conference on Intelligent Computing and Human-Computer Interaction (ICHCI), Sanya, China, 2020, pp. 337-340, doi: 10.1109/ICHCI51889.2020.00078.

[14] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "CodeSearchNet Challenge: Evaluating the State of Semantic Code Search," arXiv:1909.09436, 2019

[15] code-search-net, "code_search_net," Hugging Face, 2019. [Online]. Available: https://huggingface.co/datasets/code-search-net/code_search_net

10 Figures

Type	Description	Expected	GoCP	TF-IDF (full)	TF-IDF (kw.)	Jaccard sim.
Type-1	Identical (Kadane)	clone	✓ (1.00)	✓ (0.93)	✓ (1.00)	✓ (1.00)
Type-1	Docstring added	clone	✓ (0.91)	✓ (0.93)	✓ (0.92)	✓ (0.88)
Type-1	Inline comments added	clone	✓ (1.00)	✓ (0.91)	✓ (0.98)	✓ (0.76)
Type-1	Extra blank lines	clone	✓ (0.98)	✓ (0.94)	✓ (0.97)	✓ (1.00)
Type-1	Block comment before function	clone	✓ (1.00)	✓ (0.93)	✓ (0.98)	✓ (0.80)
Type-1	Identical (bubble sort)	clone	✓ (0.98)	✓ (0.94)	✓ (0.99)	✓ (1.00)
Type-1	Docstring + inline comments	clone	✓ (0.90)	✓ (0.94)	✓ (0.97)	✗ (0.59)
Type-1	Identical (linear search)	clone	✓ (1.00)	✓ (0.91)	✓ (1.00)	✓ (1.00)
Type-1	Mixed blank lines and comments	clone	✓ (1.00)	✓ (0.92)	✓ (0.98)	✓ (0.83)
Type-1	Identical (fibonacci)	clone	✓ (1.00)	✓ (0.92)	✓ (1.00)	✓ (1.00)
Type-2	Renamed vars (Kadane)	clone	✓ (1.00)	✓ (0.93)	✓ (1.00)	✗ (0.23)
Type-2	Renamed vars + function (binary search)	clone	✓ (1.00)	✓ (0.87)	✓ (0.99)	✗ (0.41)
Type-2	Renamed vars (bubble sort)	clone	✓ (0.98)	✓ (0.87)	✓ (0.99)	✗ (0.41)
Type-2	Renamed vars (GCD)	clone	✓ (1.00)	✓ (0.94)	✓ (0.98)	✗ (0.26)
Type-2	Renamed vars (two sum)	clone	✓ (0.98)	✓ (0.93)	✓ (1.00)	✗ (0.26)
Type-2	Renamed vars (factorial)	clone	✓ (0.98)	✓ (0.91)	✓ (1.00)	✗ (0.56)
Type-2	Renamed vars (linear search)	clone	✓ (1.00)	✓ (0.92)	✓ (1.00)	✗ (0.37)
Type-2	Renamed vars (count pairs)	clone	✓ (1.00)	✓ (0.95)	✓ (0.98)	✗ (0.35)
Type-2	Renamed vars (is_prime)	clone	✓ (0.98)	✓ (0.85)	✓ (0.97)	✓ (0.68)
Type-2	Renamed vars (fibonacci)	clone	✓ (1.00)	✓ (0.85)	✓ (1.00)	✗ (0.43)
Type-3	Guard statement added	clone	✓ (0.88)	✓ (0.93)	✓ (0.91)	✓ (0.96)
Type-3	Extra early return added	clone	✗ (0.03)	✓ (0.94)	✓ (0.96)	✓ (0.88)
Type-3	Swap assignment reordered	clone	✓ (0.58)	✓ (0.95)	✓ (0.99)	✓ (0.88)
Type-3	Extra variable for readability	clone	✗ (0.02)	✓ (0.90)	✓ (0.97)	✓ (0.80)
Type-3	Extra accumulator initialised earlier	clone	✗ (0.07)	✓ (0.90)	✓ (1.00)	✓ (0.76)
Type-3	Bounds differ by 1 (equivalent)	clone	✓ (1.00)	✓ (0.91)	✓ (0.99)	✓ (0.93)
Type-3	Default argument added	clone	✓ (0.94)	✓ (0.90)	✓ (1.00)	✓ (0.65)
Type-3	Else clause on loop added	clone	✓ (1.00)	✓ (0.96)	✓ (0.84)	✓ (0.86)
Type-3	Explicit abs() added for robustness	clone	✗ (0.16)	✓ (0.92)	✓ (0.98)	✓ (0.86)
Type-3	Max subarray with result variable	clone	✓ (0.83)	✓ (0.92)	✓ (1.00)	✓ (0.94)
Type-4	For vs while (primality)	clone	✗ (0.04)	✓ (0.82)	✓ (0.90)	✗ (0.35)
Type-4	Recursive vs iterative (fibonacci)	clone	✓ (0.44)	✓ (0.81)	✓ (0.71)	✗ (0.12)
Type-4	Recursive vs iterative (factorial)	clone	✗ (0.01)	✓ (0.83)	✓ (0.71)	✗ (0.24)
Type-4	Recursive vs iterative (GCD)	clone	✓ (0.79)	✓ (0.93)	✓ (0.98)	✗ (0.41)
Type-4	List comprehension vs for loop (filter)	clone	✓ (0.83)	✓ (0.94)	✓ (0.99)	✗ (0.53)
Type-4	Sum via loop vs accumulate	clone	✗ (0.32)	✓ (0.90)	✓ (0.78)	✗ (0.13)
Type-4	Linear search vs index lookup	clone	✗ (0.03)	✓ (0.94)	✓ (0.89)	✗ (0.50)
Type-4	Max element: loop vs built-in style	clone	✗ (0.29)	✓ (0.87)	✓ (0.87)	✗ (0.45)
Type-4	Palindrome: slice vs loop	clone	✗ (0.06)	✓ (0.77)	✓ (0.82)	✗ (0.12)
Type-4	Count zeros: sum vs loop	clone	✓ (0.57)	✓ (0.94)	✓ (0.99)	✗ (0.44)
False	Binary search vs fibonacci	not clone	✓ (0.06)	✗ (0.73)	✗ (0.66)	✓ (0.16)
False	Bubble sort vs count pairs	not clone	✗ (0.59)	✗ (0.88)	✗ (0.91)	✓ (0.20)
False	Formula vs nested loop	not clone	✗ (0.78)	✗ (0.73)	✗ (0.91)	✓ (0.13)
False	Factorial vs is_prime	not clone	✓ (0.16)	✗ (0.76)	✗ (0.75)	✓ (0.35)
False	GCD vs two sum	not clone	✓ (0.32)	✗ (0.81)	✗ (0.59)	✓ (0.10)
False	Palindrome check vs bubble sort	not clone	✗ (0.51)	✗ (0.81)	✗ (0.52)	✓ (0.12)
False	Fibonacci vs count pairs	not clone	✗ (0.61)	✗ (0.80)	✗ (0.82)	✓ (0.21)
False	Linear search vs GCD	not clone	✗ (0.62)	✗ (0.76)	✗ (0.78)	✓ (0.09)
False	Kadane vs is_prime	not clone	✗ (0.45)	✗ (0.63)	✗ (0.75)	✓ (0.16)
False	Two sum vs factorial	not clone	✓ (0.19)	✗ (0.81)	✗ (0.85)	✓ (0.20)

Figure 1: Qualitative results for GoCP, TF-IDF (Full), TF-IDF (Keywords), and Jaccard Similarly across clone types and false pairs (10 per category totalling 50 cases). Trained decision thresholds: GoC 0.44, TF-IDF (Full) 0.63, TF-IDF (Keywords) 0.49, Jaccard Similarity 0.60. A checkmark indicates a pass whilst a cross indicates a failure. The float in parentheses indicates the model's similarity score.

Clone Type	GoCP (Graph-of-Code)	TF-IDF (full)	TF-IDF (keywords)	Jaccard similarity
Type-1	10/10 (100%)	10/10 (100%)	10/10 (100%)	9/10 (90%)
Type-2	10/10 (100%)	10/10 (100%)	10/10 (100%)	1/10 (10%)
Type-3	6/10 (60%)	10/10 (100%)	10/10 (100%)	10/10 (100%)
Type-4	4/10 (40%)	10/10 (100%)	10/10 (100%)	0/10 (0%)
False	4/10 (40%)	0/10 (0%)	0/10 (0%)	10/10 (100%)
Overall	34/50 (68%)	40/50 (80%)	40/50 (80%)	30/50 (60%)

Figure 2: Proportion of correct predictions by clone type for each model, derived from Figure 1.

Metric	GoCP (Graph-of-Code)	TF-IDF (Full)	TF-IDF (Keywords)	Jaccard Similarity
Accuracy	0.8864 ± 0.0032 *	0.8707 ± 0.0029	0.8196 ± 0.0044	0.7693 ± 0.0000
Precision	0.8556 ± 0.0037	0.8585 ± 0.0026*	0.8326 ± 0.0026	0.7693 ± 0.0000
Recall	0.9306 ± 0.0045	0.9962 ± 0.0011	0.9581 ± 0.0030	1.0000 ± 0.0000 *
F1 Score	0.8915 ± 0.0030	0.9222 ± 0.0016 *	0.8909 ± 0.0027	0.8696 ± 0.0000
AUC_ROC	0.9510 ± 0.0014	0.9582 ± 0.0021 *	0.8219 ± 0.0063	0.7618 ± 0.0057
End-to-End Prediction Time (ms)	109.84 ± 38.57	146.98 ± 4.05	142.84 ± 6.11	0.17 ± 0.03

*Figure 3: Cross-validation performance of all four models across six metrics (mean ± standard deviation, 10-fold stratified cross-validation, sampled 100,000 function pairs) — * indicates best per metric based on Wilcoxon sign-rank test (w=0, p=0.002, r=0.98, n=10 fold)*